

Лабораторная работа №3.

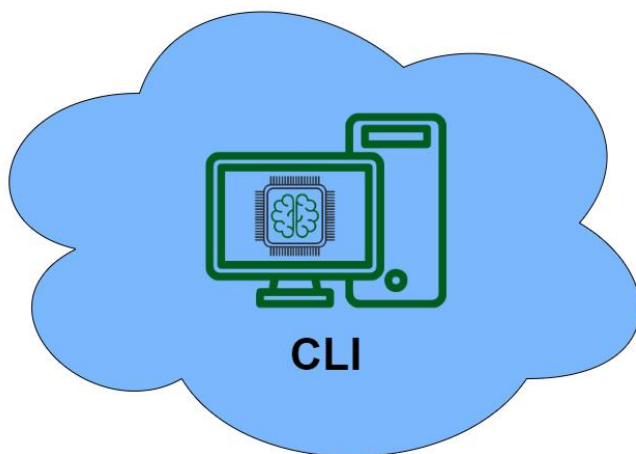
Основы работы в командной строке

Цель работы:

- Изучить основы работы в командной строке

Оборудование, ПО:

- ALT1
- Справочная литература или доступ в сеть интернет.



Машина	IP	Маска	Шлюз	DNS
CLI (Альт Рабочая станция 10.4)	- 192.168.1.1	- 24	- -	- 77.88.8.8

Контрольные задания:

1. Посмотрите текущее время и дату на Вашей системе.
2. Посмотрите календарь этого месяца.
3. Посмотрите, в какой директории вы сейчас находитесь.
4. Перейдите в Вашу домашнюю директорию.
5. Посмотрите содержимое вашего рабочего каталога.
6. Посмотрите все файлы в текущей директории, включая скрытые.
7. Отобразите список содержимого текущего каталога в широком формате.
8. Перейдите из текущей директории в директорию /etc.
9. Посмотрите содержимое файла group (данный файл находится в директории /etc)
10. Посмотрите содержимое файлов passwd и group одной командой.
11. Перейдите в Вашу домашнюю директорию. Используя шаблон, создайте N текстовых файлов (N – номер Вашего стенда), начинающихся с «stud» и

заканчивающиеся на число от 0 до N с расширением txt.

12. Создайте вложенные директории dir1/dir2/dir3.

13. Скопируйте содержимое директории /home/ в созданную директорию dir2.

14. Создайте в домашней директории пользователя новый файл с именем «alt», а затем переименуйте его в «linux».

15. Как удалить командой rm директорию dir1/dir2, если она не пуста?

16. Создайте директорию tmp_file. Используя шаблон, скопируйте все файлы с расширением txt в директорию tmp_file.

17. Используя шаблон, удалите все файлы в директории tmp_file.

18. Выведите последние 15 выполненных команд.

19. К какому типу принадлежит команда cd

20. К какому типу принадлежит команда lsblk

21. С какими параметрами будет выполнена команда ls когда пользователь набирает ls в приглашении командного интерпретатора

Используемые источники:

- Официальная документация операционной системы Альт Рабочая Станция 10.4 <https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html/alt-workstation/index.html>

Чтобы сделать работу на компьютере более продуктивной и удобной, разработчики придумали графический интерфейс. Сейчас вы к нему уже привыкли, но в начале компьютерной эры весь диалог пользователя с компьютером мог осуществляться только с помощью командной строки.

С середины XX в. началась разработка удобной системы взаимодействия с компьютером. Главной целью было создание такой операционной оболочки, с которой обычным пользователям было бы удобно работать.

Первый полноценный графический интерфейс появился у компьютеров Apple (OS Apple Lisa) и IBM (OS Windows). Компании позаимствовали идею графического интерфейса у Xerox. Появление удобной оболочки для работы с компьютером стало переломным моментом в истории IT. Apple и Microsoft до сих пор конкурируют и борются за своего пользователя. Позже набрала популярность и ОС Linux, у которых графический интерфейс появился в 1990-х гг.

Пользовательский интерфейс — средства удобного и эффективного взаимодействия пользователя с устройствами компьютера.

Основные типы пользовательского интерфейса:

1. Графический интерфейс пользователя (Graphical User Interface, GUI)

GUI (Graphical User Interface) - приложением или графическим приложением называют практически любую программу, управление которой осуществляется посредством мыши, тачпада или сенсорного экрана. Графический интерфейс предлагает визуальное взаимодействие с системой через элементы, такие как окна, иконки и меню. GUI делает систему более интуитивно понятной и доступной для широкого круга пользователей. В Linux возможность графически взаимодействовать с операционной системой предоставляется так называемой средой рабочего стола. Поверх неё можно пользоваться GUI-приложениями, такими как GIMP, VLC, Firefox, LibreOffice, файловый менеджер и т.д. для решения различных задач.



2. Интерфейс командной строки (Command line interface, CLI)

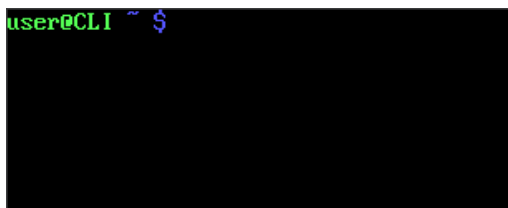
CLI (Command Line Interface) – программа в командной строке, которая читает вводимые команды и выполняет запрошенные действия. В целом, любая программа, которой можно пользоваться через команды из терминала, попадает

в эту категорию. CLI позволяет пользователям вводить текстовые команды напрямую в терминал или консоль для выполнения операций в системе.

У старых компьютеров не было мыши или других манипуляторов для управления операционной системой. Всё взаимодействие выполнялось через текстовый ввод и вывод, т.е. через CLI.

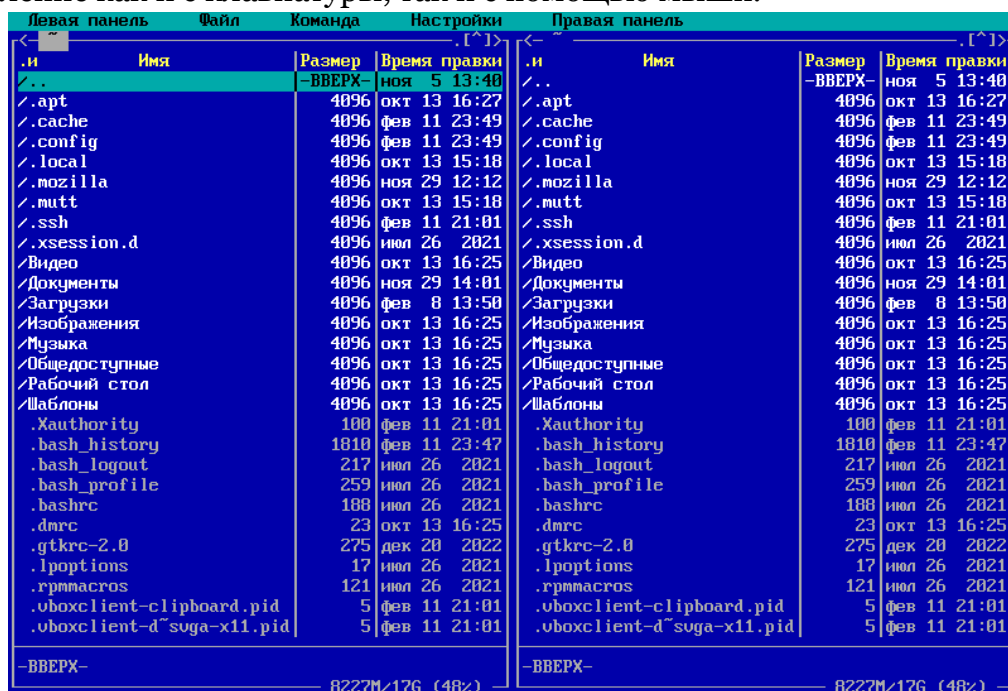
Ушёл ли CLI в прошлое? Вовсе нет. У командной строки есть свои преимущества, особенно если речь идёт о настройке Linux на глубинном уровне: поднятии сетевого брандмауэра, организации локальной сети или даже управлении пакетами.

Со всеми этими вопросами можно разобраться и с помощью GUI-программ, но CLI даёт более прямой и прицельный доступ к операционной системе. В любом случае, работа GUI-приложений также основана на обращении к ОС с помощью команд (которые прописаны в их программном коде).



3. ТПИ - текстовый пользовательский интерфейс (Text user interface, TUI; также Character User Interface, CUI)

Как было сказано выше, работа со старыми компьютерами была целиком основана на CLI. Однако некоторый ограниченный спектр возможностей GUI можно имитировать и в текстовом терминале. Это и будет TUI: в отличие от интерфейса командной строки, в нём представлено больше виджетов и доступно управление как и с клавиатуры, так и с помощью мыши.



Основным интерфейсом взаимодействия пользователя с операционной системой Linux во всех ее модификациях была и остается *командная строка*. Суть этого интерфейса состоит в *обмене текстами* с системой: пользователь набирает текстовые команды, а операционная система возвращает ему информацию также в текстовом виде. Несмотря на то, что в современных linux-системах широко используются графические пользовательские интерфейсы, командная строка остаётся основным рабочим инструментом администратора системы. Причина этого не в каких-то особенных вкусах администраторов, а в том, что в UNIX-подобных системах командная строка очень хорошо выполняет задачи интерфейса управления системой, она эффективна и удобна — и именно поэтому всеми используется.

Командный интерпретатор

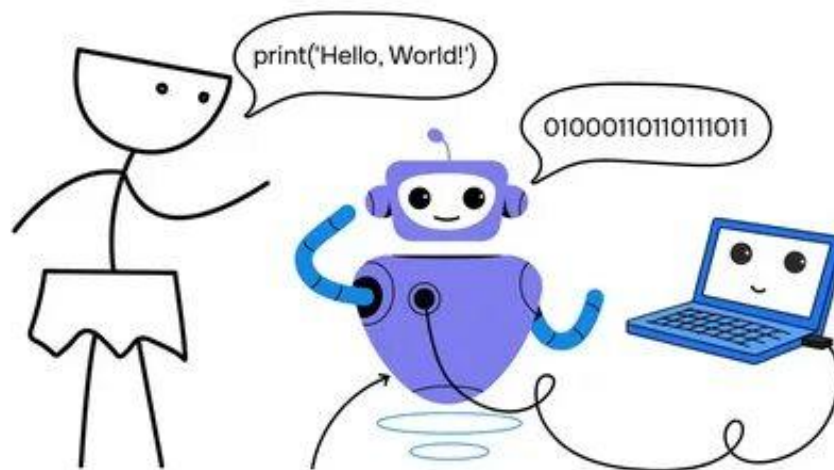
Командный интерпретатор, интерпретатор командной строки — компьютерная программа, часть операционной системы, обеспечивающая базовые возможности управления компьютером посредством интерактивного ввода команд через интерфейс командной строки или последовательного исполнения пакетных командных файлов.

Как правило его функции сводятся к предоставлению пользователю возможности запускать другие программы, может также содержать некоторые базовые команды ввода-вывода и свой простой скриптовый язык программирования.

Как правило, при низкоуровневой настройке ОС у пользователя есть возможность менять командный интерпретатор, используемый по умолчанию.

К функциям интерпретатора командной строки относятся:

1. Взаимодействие с пользователем (редактирование командной строки, история команд и т. д.).
2. Обработка (расширение) шаблонов имен («*», «?» и т. д.).
3. Перенаправление ввода-вывода команд.
4. Управление заданиями.



При работе в интерактивном режиме интерпретатор может находиться в двух состояниях.

В состоянии ожидания команды интерпретатор ничего не делает, только ждёт, пока пользователь введёт нужные инструкции и нажмёт Enter.

После этого интерпретатор переходит в состояние выполнения команды. В это время ничего вводить нельзя (разве что команда запросит какие-то данные), а интерпретатор занят выполнением команд, которые пользователь ввёл на предыдущем шаге.

Для удобства пользователя командная строка оформляется в виде приглашения — специальной последовательности символов, которую командная оболочка выводит на терминал в знак того, что она готова принять очередную команду. Заодно приглашение командной строки служит целям визуальной организации интерфейса пользователя: прежде всего, она указывает область экрана, где будет отображаться вводимая пользователем команда (непосредственно справа от приглашения, в той же строке). Кроме того, в приглашении принято помещать информацию о некоторых текущих свойствах командной оболочки (имени пользователя, текущем каталоге и т. п.) — такая информация выполняет роль «подсказок» пользователю, которые облегчают ориентацию в интерфейсе.

Пример выполнения команды:

```
user@altworkstation ~ $ date
Cp 29 мая 2024 21:53:02 MSK
user@altworkstation ~ $
```

Здесь приглашением является строка `user@altworkstation ~ $`, которая содержит: имя пользователя, с правами которого выполняется командная оболочка (`user`), имя компьютера (`altworkstation`), имя текущего каталога (`~`, этим символом в командных оболочках UNIX принято обозначать домашний каталог пользователя). Заканчивается приглашение символом начала ввода (`$`), который, по существу, является единственным обязательным элементом приглашения и показывает собственно готовность командной оболочки ко вводу команды. В качестве символа начала ввода могут выступать разные символы, но чаще всего используется именно (`$`) для обозначения командной строки обычного пользователя и символ (`#`) для обозначения командной строки суперпользователя (`root`). Пользователь имеет возможность произвольным образом изменить оформление приглашения командной строки.

Откройте терминал на рабочей станции. Распишите, что значит каждый параметр в приглашении командной строки

[illegible]

Командные интерпретаторы в ОС Альт

sh - Bourne Shell

Bourne shell (часто sh по имени исполняемого файла) — ранняя командная оболочка UNIX, разработанная Стивеном Борном из Bell Labs и выпущенная в составе 7-го издания операционной системы UNIX (UNIX Version 7). Данная оболочка является де-факто стандартом и доступна почти в любом дистрибутиве Unix. Существует много командных оболочек, основанных (идейно или напрямую) на Bourne shell.

bash - Bourne again shell

Bourne again shell, самая популярная на данный момент для ОС Linux, является модернизированной версией оболочки sh, распространяется свободно. Bash (от англ. Bourne again shell, каламбур «Born again» shell — «возрождённый» shell) — усовершенствованная и модернизированная вариация командной оболочки Bourne shell. Одна из наиболее популярных современных разновидностей командной оболочки UNIX. Особенно популярна в среде Linux, где она часто используется в качестве предустановленной командной оболочки.

Представляет собой командный процессор, работающий, как правило, в интерактивном режиме в текстовом окне. Bash также может читать команды из файла, который называется скриптом (или сценарием). Как и все Unix-оболочки, он поддерживает автодополнение имён файлов и каталогов, подстановку вывода результата команд, переменные, контроль над порядком выполнения, операторы ветвления и цикла. Ключевые слова, синтаксис и другие основные особенности языка были заимствованы из sh.

csh - C shell

C shell (csh) — командная оболочка UNIX со встроенным скриптовым языком, разработанная Биллом Джоем, активным разработчиком BSD UNIX и создателем редактора vi, в 1979 году.

Базировался на коде командного интерпретатора шестой версии UNIX. Скриптовый язык не уступает Bourne shell по мощности, но отличается синтаксисом. В то время как Борн скопировал все основные операторы с языка Алгол 68, Билл Джой использовал в качестве макета язык Си, вероятно, руководствуясь своими предпочтениями и предпочтениями других пользователей BSD UNIX.

В начале 1990-х годов C shell подвергся критике за свою двусмысленность и немногословность интерпретатора, останавливающего выполнение скрипта, но не сообщаящего никаких подробностей о том, что же всё-таки произошло. Порой скрипты csh работали совсем не так, как этого ожидал пользователь. Также встречались ситуации, когда интерпретатор отбраковывал, казалось бы, непротиворечивые строки кода.

ksh - Korn shell

Korn shell (ksh) — командная оболочка UNIX, разработана Дэвидом Корном (AT&T) в 1980-х годах. Имеет полную обратную совместимость с Bourne shell и включает в себя возможности C shell. Является стандартом де-факто и используется по умолчанию в некоторых Unix-системах, например, в AIX, OpenBSD и QNX. Имеет развитую систему горячих клавиш.

До 2000 года код оболочки был закрытым.

zsh - Z shell

Z shell, zsh — одна из современных командных оболочек UNIX, использующаяся непосредственно как интерактивная оболочка, либо как скриптовый интерпретатор. Zsh является расширенным аналогом, а также имеет обратную совместимость с bourne shell, имея большое количество улучшений.

Некоторые полезные особенности:

- программируемое автодополнение, которое помогает пользователям вводить как команды, так и их аргументы, со встроенной поддержкой нескольких сотен команд;
- разделяет историю команд между всеми запущенными экземплярами оболочки;
- расширенное дополнение названий файлов, что позволяет указать файл без необходимости запускать внешние программы, наподобие find;
- расширенная поддержка переменных и массивов;
- редактирование многострочных команд в едином буфере;
- коррекция опечаток;
- имеет различные режимы совместимости (то есть, вы можете использовать zsh вместо bourne shell при запуске, как /bin/sh);
- модифицируемые приглашения (prompts), включающие возможность расположить приглашение справа и настроить автоскрытие при наборе длинных команд;

Виртуальные консоли

Характерный для современных версий linux способ организации параллельной работы пользователей — *виртуальные консоли*. Виртуальные консоли (virtual console) — это несколько параллельно выполняемых операционной системой программ, предоставляющих пользователю возможность зарегистрироваться в системе в текстовом режиме и получить доступ к командной строке

В операционной системе Linux переключение между виртуальными консолями в текстовом режиме производится с помощью комбинации клавиш **Alt-F1**, **Alt-F2** и т. п. При этом каждая виртуальная консоль обозначается специальным именем: «tty1», «tty2» и т. д. По умолчанию в Linux

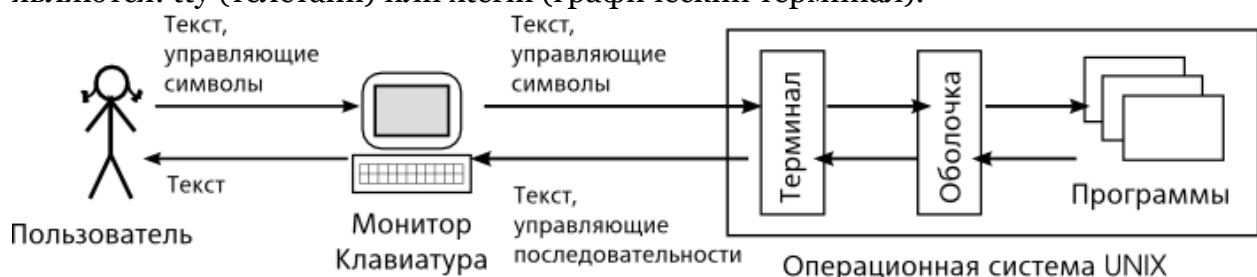
доступно не менее шести виртуальных консолей, переключаться между которыми можно при помощи сочетания клавиши **Alt** с одной из функциональных клавиш (**F1-F6**). С каждым сочетанием связана соответствующая по номеру виртуальная консоль. Виртуальные консоли обозначаются «ttyN», где «N» — номер виртуальной консоли. На самом деле, каждая из таких консолей связана с собственным *терминалом*, который характеризуется файлом устройства с соответствующим именем (например, /dev/tty1). Двенадцатая виртуальная консоль (Ctrl+Alt+F12) выполняет функцию системной консоли — на нее выводятся сообщения о происходящих в системе событиях.

TTY — файл устройства, который создается ядром и предоставляет доступ к терминалу для программ. Это могут быть файлы /dev/tty для постоянных текстовых терминалов и /dev/pts/* для эмуляторов терминалов. Вы можете выполнить команду или отправить сообщение просто записав данные в этот файл, и также получить результат, прочитав данные из этого файла.

Также существует возможность запуска нескольких графических подсистем, тогда переключение между ними будет осуществляться аналогично переключению между виртуальными консолями — комбинацией клавиш **Ctrl-Alt-F1**.

Продемонстрируйте переключение между терминальными линиями.

Каждое устройство или программа, выполняющие функции терминала, имеет свои *возможности* по вводу и выводу информации. Примерами таких возможностей являются: число цветов отображения, способность перемещать курсор и изменять размер экрана, набор управляющих символов и т. п. При этом для операционной системы важна возможность работать с любыми терминалами одинаковым способом. Для этого в операционных системах семейства UNIX вводится понятие *тип терминала*: каждый тип сводится к набору возможностей, регламентированных в специальном конфигурационном файле. Информация о всех существующих типах терминалов хранится в специализированной базе данных (terminfo или termcap). Примерами типов являются: tty (телетайп) или xterm (графический терминал).



Все управляющие символы, интерпретируемые системой, можно перенастроить с помощью команды `stty`. `Stty` происходит от “set tty” (установить терминал).

\$ stty -a – выводит полный список сигналов, которые можно перенастроить

Команда “`stty -a`” выводит текущие настройки терминала. Опция `-a` позволяет вывести все параметры, такие как режимы ввода, вывода, контроля и другие настройки, установленные для текущего терминала.

Продемонстрируйте результат работы команды

\$ reset – команда для инициализации терминала, возвращающая его в исходное состояние

Команда `reset` используется для сброса терминала в его исходное состояние.

Команды в Unix-подобных системах

Стандартный синтаксис командной строки предписывает унифицированный способ формирования команд, опций и параметров.

Введем в терминале команду `date`:

`# date`

Продемонстрируйте результат работы команды

Этот пример демонстрирует простейшую команду, которая состоит только из имени команды: `date`. Для исполнения этой команды командная оболочка находит и запускает программу, имя которой совпадает с именем команды. В данном примере это утилита `date`, которая и выводит на терминал пользователя информацию о текущей дате и времени. Некоторые команды (такие как `echo`) являются встроенными для определенной командной оболочки и для их выполнения вызывается не программа, а внутренняя функция оболочки.

Типы команд:

1. Внутренние команды командного интерпретатора
2. Внешние команды
3. Синонимы команд

Основное отличие внешней команды от внутренней заключается в порождении нового подпроцесса.

Ветвление процесса (forking) — это действие, когда либо команда, либо командная оболочка порождает новый подпроцесс, чтобы выполнить какую-либо работу. Порождающий процесс называется родительским, или предком. Порожденный — дочерним, или потомком.

После ветвления родительский и дочерний процессы продолжают выполняться параллельно (независимо по времени). Встроенные команды shell не порождают новый подпроцесс, а выполняются в текущей оболочке shell. Встроенная команда выполняется быстрее и является более производительной. Реализацию встроенной команды выполняет программа bash (исполняемый файл /bin/bash). Количество встроенных команд может быть разным в зависимости от версии командного интерпретатора bash.

Внешние программы порождают дочерний процесс и выполняются в подоболочке shell (subshell). Реализация внешней команды — это отдельный исполняемый файл (программа), который находится в директории /bin. Например, внешняя команда echo реализуется программой /bin/echo.

Команда type используется для отображения информации о типе команды.

Выполните определение типа команды tar, date, ls, cd, echo. Сделайте вывод

Встроенная команда может иметь внешние аналоги с таким же именем (например, команда echo). Как правило, интерпретатор shell по умолчанию выбирает встроенный вариант. Однако можно запретить использование внутренней команды или вызвать внешнюю команду вместо внутренней, указав полный путь к внешней программе.

Вместо команды type для вывода пути к внешней программе можно использовать команду which.

- команда **which**

Позволяет узнать, какая программа будет запущена (где будет она вызвана) при вводе данной команды (для внешних команд)

Определите путь к исполняемому файлу для программ tar, date, find. Сделайте вывод

В качестве имени команды, помимо внешней программы и встроенной функции, может выступать еще и *псевдоним* (alias).

Продемонстрируйте результат работы команды

Сокращения не наследуются с окружением.

Для создания новых псевдонимов используется команда `alias`.

alias [название алиаса] = “[значение]”

Алиас тусал выполнит команду “cal 2020”, отображая календарь на 2020 год.

Команды могут быть устроены и более сложным образом, когда необходимо не только выполнить внешнюю программу, но и передать ей какую-то дополнительную информацию, чтобы так или иначе модифицировать ее работу. Модификаторы поведения в этом случае вводятся в командной строке после имени команды — это *параметры команды*. Параметров может быть произвольное количество (оно зависит только от смысла передаваемой внешней программе информации). Друг от друга параметры отделяются пробелами. Параметры будут переданы программе в том порядке, в котором они указаны в командной строке.

Параметры, которые могут быть переданы команде, подразделяются на два вида: первые — это внешние объекты, с которыми должна работать вызываемая команда (например, имена файлов, числа), вторые — это принимаемые программой модификаторы поведения. Последовательность в командной строке обычно такая: *имя команды — модификаторы — объект(ы)*, где и модификаторы, и объекты необязательны.

Проблемы могут возникнуть, если с символа «-» начинается не ключ, а обычный параметр, например, имя файла. В этом случае команда может интерпретировать этот параметр как ключ, что повлечет ошибку. Для выхода из этой ситуации используется специальная конструкция «--», указывающая, что список ключей закончен и любые последующие строки следует интерпретировать как параметры.

```
$ ls -l -- file.txt
```

Продемонстрируйте результат работы команды

Навигация по дереву каталогов

Навигация по дереву каталогов Linux является одной из основных задач, с которой сталкиваются пользователи операционной системы на ежедневной основе. Для того чтобы успешно ориентироваться в файловой системе Linux, необходимо понимать ее структуру и основные принципы организации дерева каталогов.

Linux использует иерархическую структуру каталогов, которая начинается с корневого каталога "/" и далее разветвляется на более мелкие подкаталоги. Все файлы и каталоги в Linux располагаются внутри этой древовидной структуры.

Для навигации по дереву каталогов Linux используются команды командной строки.

Команда `ls` используется для отображения содержимого каталогов.

- **-lh**
Опция используется для вывода списка файлов с детальной информацией о каждом файле. Она отображает размер файла, разрешения доступа, количество жестких ссылок, владельца, группу, размер файла в удобочитаемом формате (например, Кб, Мб), а также дату и время последнего изменения.
- **-a**
Опция используется для вывода всех файлов, включая скрытые файлы.
- **-d**
Опция используется для отображения только имен каталогов, а не их содержимого.
- **-tr**
Опция используется для сортировки файлов по времени последнего изменения в порядке возрастания.
- **-S**
Опция используется для сортировки файлов по размеру в порядке убывания.

Продемонстрируйте результат работы команды со всеми представленными параметрами. Отдельно используйте параметр <code>-l</code> . Укажите его назначение

Команда `cd` используется для перемещения по директориям.

- `-` (минус) используется для перехода в предыдущий рабочий каталог.
- `..` (две точки) представляют родительский каталог текущего рабочего каталога.
- `~` (тильда) используется как сокращение домашнего каталога текущего пользователя.

В качестве аргумента передается путь до целевой директории.

```
$ pwd
```

Команда для вывода абсолютного пути к текущему рабочему каталогу.

Продemonстрируйте результат работы команд

Утилита touch создает пустые файлы или обновляет время модификации существующих файлов.

Пример (в терминале):

```
$ touch file1.txt
```

```
$ touch -a file1.txt
```

Продemonстрируйте результат работы команд

Утилита mkdir создает новый каталог (папку).

Основные опции:

-p создание подкаталогов, если их нет;

Пример (в терминале):

```
$ mkdir dir
```

Создаст новую папку с именем dir.

```
$ mkdir -p dir1/dir2
```

Создаст родительскую папку dir1 и вложенную дочернюю папку dir2.

Продemonстрируйте результат работы команд

Утилита `cp` копирует файлы и директории.

Основные опции:

-r рекурсивное копирование директорий;

-i предупреждает перед перезаписью существующего файла;

Пример (в терминале):

```
$ cp file.txt new_file.txt
```

Скопирует файл `file.txt` в `new_file.txt`.

```
$ cp -r dir1 dir3
```

Скопирует содержимое директории `dir1` в директорию `dir3`.

Продemonстрируйте результат работы команды

Утилита `mv` перемещает файлы и директории.

Основные опции:

-b создает резервную копию файла перед перемещением;

-i предупреждает перед перезаписью существующего файла;

Пример (в терминале):

```
$ mv file.txt dir1/dir2
```

Переместит файл `file.txt` в новое местоположение `dir1/dir2`.

```
$ mv -b new_file.txt dir3
```

Создаст резервную копию файла `file.txt` и переместит ее в папку `dir3` (РЕЗЕРВНОГО ФАЙЛА НЕТ).

Продemonстрируйте результат работы команды

Утилита `rm` удаляет файлы и директории.

Основные опции:

-r рекурсивное удаление директорий;

-f принудительное удаление без запросов подтверждения;

Пример использования:

```
$ rm file1.txt
```

Удалит файл `file.txt`.

```
$ rm -r dir1/
```

Удалит директорию `dir1` со всем ее содержимым.

Продемонстрируйте результат работы команды

Шаблоны

Шаблон в командном интерпретаторе используется примерно в тех же целях, что и регулярное выражение. В отличие от регулярного выражения, шаблон всегда примеряется к строке целиком, кроме того, он устроен значительно проще.

Символы в шаблоне разделяются на обычные и специальные. Обычные символы соответствуют таким же символам в строке, а специальные — обрабатываются особым образом:

1. Шаблону, состоящему только из обычных символов, соответствует единственная строка, состоящая из тех же символов в том же порядке. Например, шаблону «`abc`» соответствует строка `abc`, но не `aBc` или `ABC`, потому что большие и маленькие буквы различаются.

Пример (в терминале):

```
$ ls file
```

Команда выведет список файлов, включающих в название строку «`file`».

2. Шаблону, состоящему из единственного спецсимвола «`*`», соответствует любая строка любой длины (в том числе и пустая).

Пример (в терминале):

```
$ ls *.txt
```

Команда выведет список всех файлов с расширением `.txt` в текущем каталоге.

Продемонстрируйте результат работы команд

--

3. Шаблону, состоящему из единственного спецсимвола «?», соответствует любая строка длиной в один символ, например, а, + или @, но не ab или 8888.

Пример (в терминале):

```
$ touch file2 file3
```

```
$ cp file? /dir3
```

Команда копирует файлы с названием, начинающимся с file и имеющим один дополнительный символ, в директорию dir3.

Продemonстрируйте результат работы команд

4. Шаблону, состоящему из любых символов, заключенных в фигурные скобки «[» и «]» соответствует строка длиной в один символ, причем этот символ должен встречаться среди заключенных в скобки. Например, шаблону «[bar]» соответствуют только строки a, b и r, но не c, B, bar или ab. Символы внутри скобок можно не перечислять полностью, а задавать диапазон, в начале которого стоит символ с наименьшим ASCII-кодом, затем следует «-», а затем — символ с наибольшим ASCII-кодом. Например, шаблону «[0-9a-fA-F]» соответствует одна шестнадцатеричная цифра (скажем, 5, e или C). Если после «[» в шаблоне следует «!», то ему соответствует строка из одного символа, не перечисленного между скобками.

Пример (в терминале):

```
$ touch doc{o..4}
```

Продemonстрируйте результат работы команд

Команда создаст новые файлы с именами doco, doc1, doc2, doc3, doc4.

```
$ ls doc[0-4]
```

Команда выведет список файлов, начинающимся с «doc» и завершающимся цифрами от 0 до 4.

Продемонстрируйте результат работы команд

5. Шаблону, состоящему из нескольких частей, соответствует строка, которую можно разбить на столько же подстрок (возможно, пустых), причем первая подстрока будет отвечать первой части шаблона, вторая — второй и т. д.

Пример (в терминале):

```
$ rm doc{3,4}
```

Команда удалит все файлы с названием, начинающимся с «doc» и завершающимся 3 и 4.

Продемонстрируйте результат работы команд

История команд

Все команды, набранные пользователем, командная оболочка запоминает и позволяет обращаться к ним впоследствии. По стрелке вверх, список поданных команд «прокручивается» от последней к первой, а по стрелке вниз — обратно. Соответствующая команда отображается в командной строке как только что набранная, её можно отредактировать и подать оболочке (подгонять курсор к концу строки при этом необязательно).

Чтобы история команд могла сохраняться между сеансами работы пользователя, `bash` записывает ее в файл `.bash_history`, находящийся в домашнем каталоге пользователя. Делается это в момент завершения оболочки: накопленная за время работы история дописывается в конец этого файла. При следующем запуске командной оболочки считывает `.bash_history` целиком. История хранится не вечно, количество запоминаемых команд в `.bash_history` ограничено (обычно сохраняется 500 команд, но это можно и перенастроить).

Команда `history` используется для отображения списка выполненных команд в текущем сеансе терминала. Она выводит список предыдущих команд, включая номера команд.

```
$ history
```

